

Week #01

Basic Drawing Functions

- `ellipse(x, y, w, h)`
- `line(x1, y1, x2, y2)`
- `point(x, y)`
- `quad(x1, y1, x2, y2, x3, y3, x4, y4)`
- `rect(x, y, w, h)`
- `triangle(x1, y1, x2, y2, x3, y3)`
- `size(x,y)`

Example:

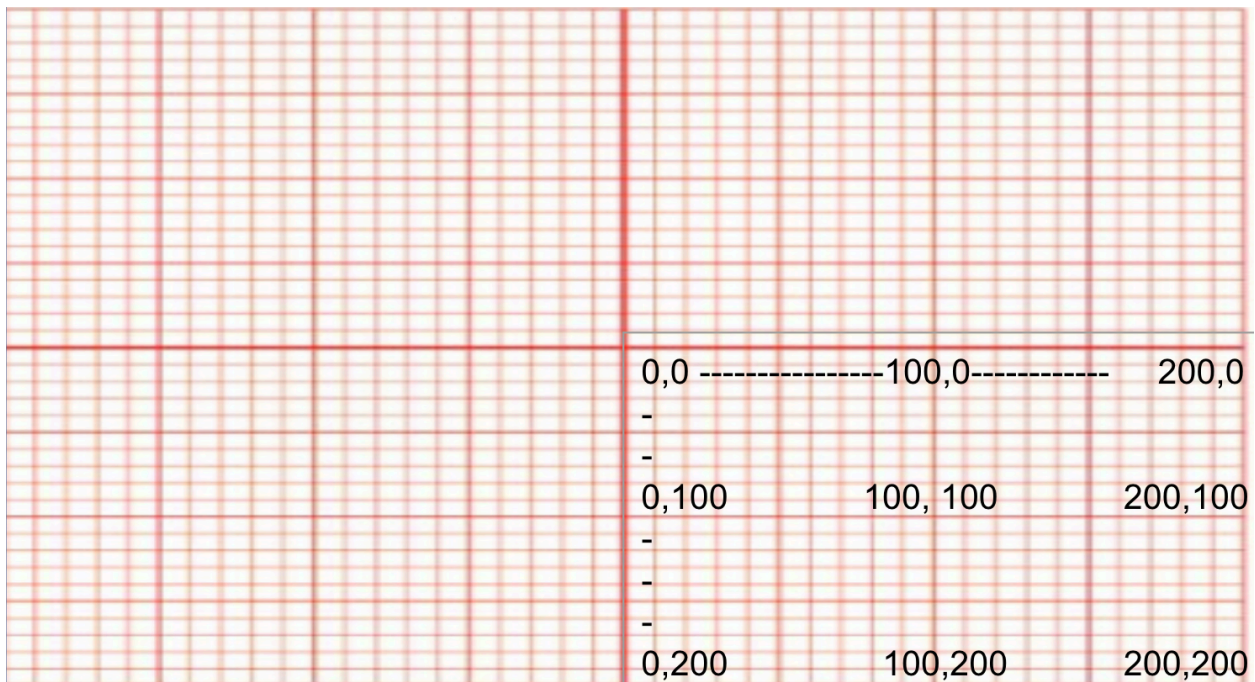
```
size(200,200); // define the dimension of the canvas
```

```
rect(50,25,75,25);    // draw a rectangle
```

```
ellipse(100,100,25,25);    // a circle
```

```
ellipse(150,75,50,25);    // an oval
```

```
triangle(50,150,50,175,100,135);    // a triangle
```



Notes

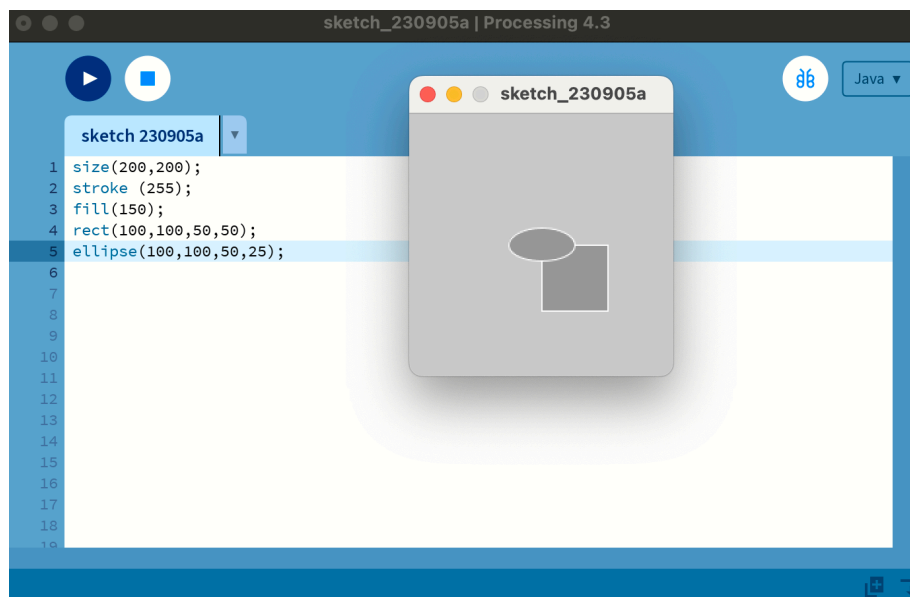
- Drawing commands are called **functions**.
- Most functions have a set of **arguments** that MUST be matched with correct values.
 - For example:
`rect(50, 200, 50, 100);`
i.e., x= 50, y=200, w= 50 & h = 100
- Each command statement must end with a semicolon

Intro to Drawing:

Most basic instructions are **function calls**. They are used to “**Invoke**” the functions.

- Each program has a set of instructions that must be correctly coded. The following code block has multiple common errors. Copy, run, and debug it. Please pay attention to the error messages

```
size(200,200);  
stroke 255;  
fill(150)  
rect(100,100,50,50);  
ellipse(100,100,50,25);
```



Exercise 1:

Open a new sketch and complete code described below

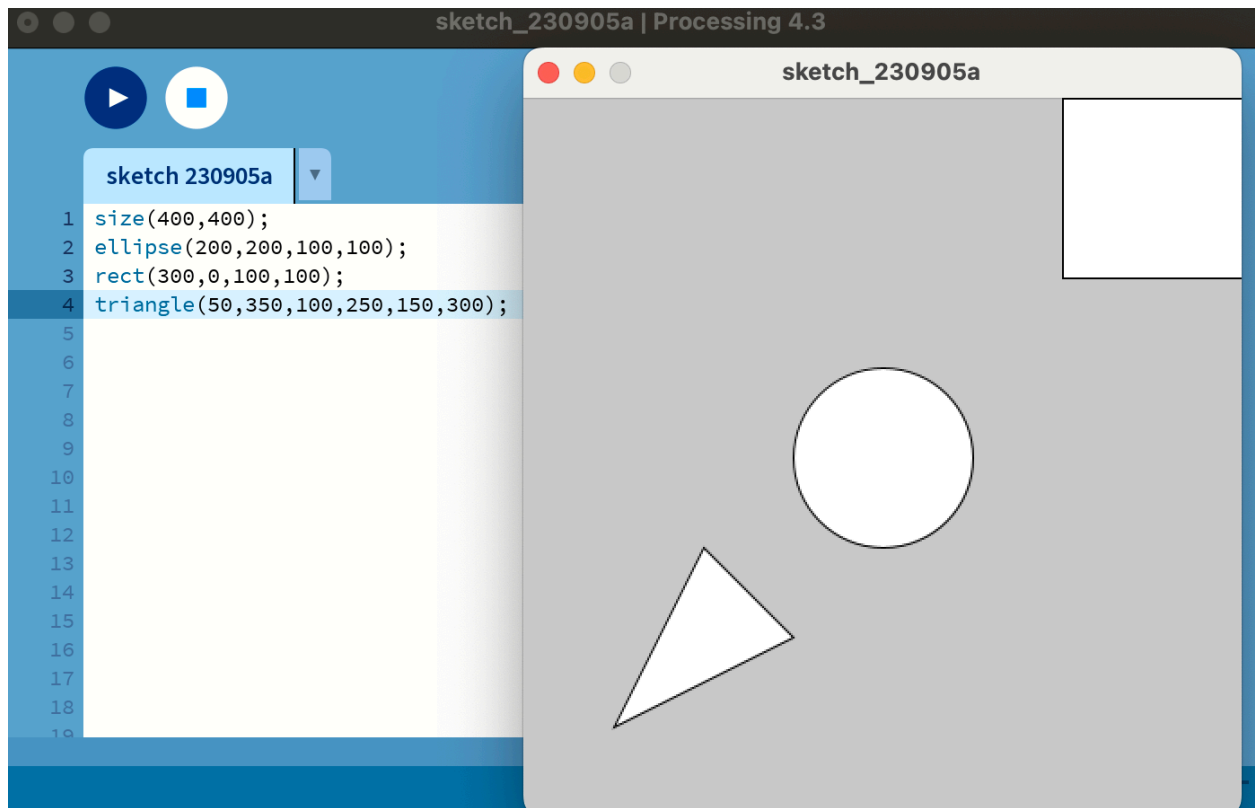
```
size(400,400);
```

```
ellipse(x, y, w, h); //replace x, y, w, h to draw a circle at the center
```

```
_____ // draw a rectangle in the top right quadrant
```

```
_____ // draw a triangle in the bottom left quadrant
```

Run the sketch on Processing to verify. Debug as needed



Chapter 1: Drawing Modes: Attributes

The default

- **rect(x,y,w,h):** x & y refer to the upper- left corner of the rectangle
- **ellipse(x,y,w,h):** x & y refer to the center of the oval

Use rectMode(mode) and ellipseMode(mode) to re-define the anchored point

Mode setting (attributes):

CENTER: set x,y coordinate at the center of the shape

CORNER: set x,y coordinate at the top-left

CORNERS: The four values are the top and bottom right corner--No width or height

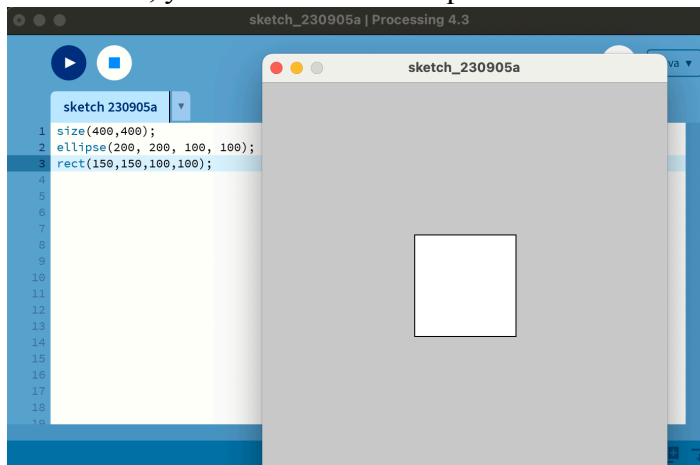
RADIUS: same as CENTER. Instead w & h are radiuses (of the ellipse)

Processing: Intro to Drawing:

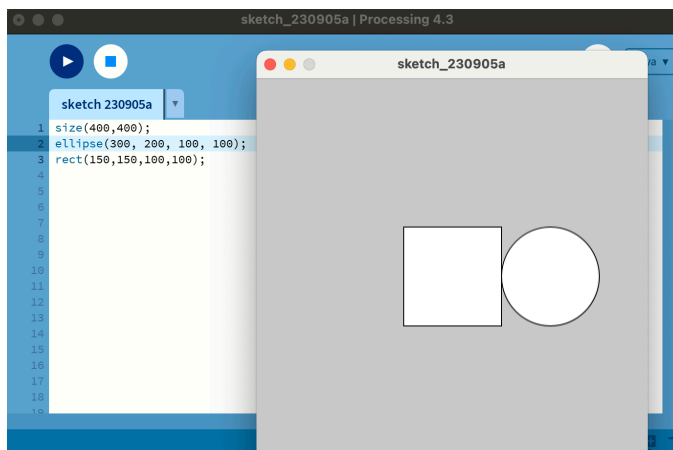
Exercise 2: Given the code block below:

```
size(400,400);  
ellipse(200, 200, 100, 100);  
rect(0,0,100,100);
```

- Move the x, y coordinates of the square to where the circle is at.



- Are they at the same location? Try using rectMode function
- See the circle? How would you fix it?
- Move the circle in front of the rectangle



Exercise #3:

Complete this **toyCar** program to draw a simple two-wheel toy car as shown. Save it as *lastnameToyCar*

- Always add your LASTname to the file name (required)
- Follow this file naming scheme:
 - No special characters & No spaces
 - Not to begin with a number
 - Capitalize the first letter of the subsequent word(s)
 - See Examples here
- ALSO Strongly Recommend: Use a separate folder each week

Setting & Environment:

- **stroke(color)** or **noStroke()**
- **fill(color)** or **noFill()**
- **size(w, h)**
- **background(color)**
- **strokeWeight(point)**

Others: See Reference: <https://processing.org/reference/>

Note that the first letter of secondWord in a command is capitalized. Examples:

noFill();
strokeWeight(n);

* some functions, e.g., noStroke() and noFill(), do not have arguments

Basic RGB Color Properties

RGB color & transparency: Ref: <https://processing.org/reference/>

- Basic color scheme uses 3 bytes (24 bits) to store each colored pixel
- Each byte (8 bits) is used to store one of the Red, Green, and Blue element. Collectively labelled as RGB
- In binary representation we have:

00000000 = 0

00000001 = 1

11111111 = 255

Thus a byte can store 256 shades or variations of each color element.

- Together a 24 bit RGB color yields $256 \times 256 \times 256 = 16.7$ million possible colors

Implementation of RGB color

To fill a shape with color, we need to use one of the color-arguments as described below:

- One number: from 0 (black) to 255 (white) for a total of 256 possible grey shades
`fill(0);` // fill the drawing black
- Two numbers: one for B/W color & the other for opacity
`fill(150, 100);` // the 100 is Opacity
- Three numbers: a number for each R, G, and B color
`fill(250,250,0);` //fill shape yellow
- Four numbers: RGB and Opacity
`fill(250,0,0, 125);` // red, 50% opacity
- Use EX2 and implement colors

More Examples:

Explore and learn from the following examples @ learningProcessing.com

- **Example_1.1** Stroke/NoStroke
- **Example_1.2** Fill/NoFill
- **Example_1.3** (RGB color)
- **Example_1_4** (Opacity)
- **Example_1.5** (Zoog)

Week #02

*/ Sample program formatting style.

- 1) Name
- 2) Program statement,
- 3) Comment
- 4) indentation

*/

#1 Name: Student name

#2 Program Statement: This program draws Zoog as dynamic sketch

#3 setup() function executes one time at the beginning.

void setup(){

// #4 shows indentation of code by level} // end of setup

Arc (and similarly curve): is used to draw a section of an oval

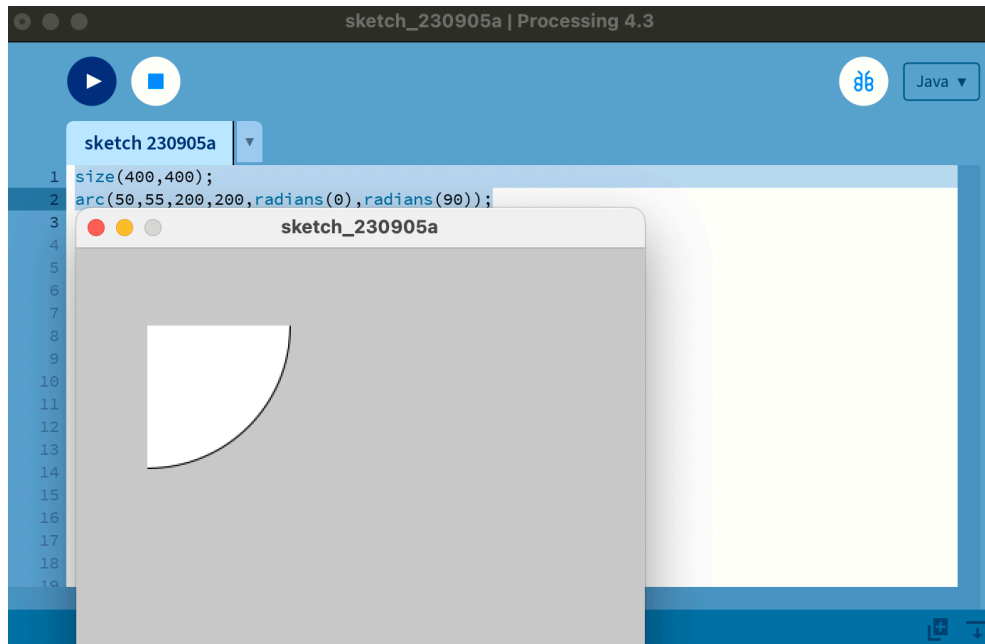
- **arc(x,y,w,h,start,stop,mode*)**
-x,y,w,h: (same as ellipse)
-start, stop: radians(degrees) where:
 start = start point (degree)
 stop = end point (start + angle);
-start/stop can be a constant of PI:
 PI, HALF_PI, QUARTER_PI...
*(optional) mode: OPEN, CHORD, PIE

- **Coding Examples: [Reference](#)**

***or instead you can use radians(x) function**

size(400,400);

arc(50,55,200,200,radians(0),radians(90));



Exercise

Complete this Pie Chart program and save it. You will need this for Lab #1

Drawing a Curve

Bezier

- **bezier(x1,y1, cx1,cy1, cx2,cy2, x2,y2)**
 -x1,y1: where the curve begins
 -cx1,cy1: control point 1
 -cx2,cy2: control point 2
 -x2,y2: where the curve ends
- **Try some of these coding examples (Reference)**

** Alternately you may use curve function instead of bezier*

Exercise:

```
size(400,400);
// draw a straight line as Ys are equal
bezier(100,100, 150,100, 200,100, 250,100);
//now change Y of the control points
// also change X of the control points
```

Exercise:

Complete this drawALeaf program and save it

Chapter 2: Function & Interaction:

We have learned and implemented various built-in functions:

2D primitives:

`rect(arguments);`

`ellipse(arguments);`

Attributes & environment:

`rectMode(argument);`

`fill(R,G,B);`

`noStroke();`

So what is a function?

Generalization: A block of executable and **reusable** code that performs a specific task.

- Some functions require certain values, called arguments. For example, `rect(100,100,50,40);`
- Other functions do not have any such as `noFill(); noStroke();`

Compare Example 3.1 to example 2.1, the 3.1 program is organized into two functions: a setup function and a draw function.

- `void setup()`
 `setup()` is used for setting up the sketch environment and commands that **only execute one time**.
- `void draw()`
 `draw()` is a built-in function that repeatedly executes the code inside the function, i.e., reusing of code

void: for now, void is the function keyword that precedes function name. We will discuss this later

Also code inside the function are indented and enclosed in { } pairs

Processing has many built-in **system variables**. Among them:

mouseX, mouseY : use the current x,y location of the mouse cursor

pmouseX, pmouseY: use the previous x,y location of the mouse

Consider example 3.2: The program uses mouseX and mouseY as x and y coordinates of the rect. When the mouse moves, mouseX and mouseY changes, so is the rect.

Also study example 3.3 and 3.4

Exercise:

Complete Example 3.3 to move Zoog. Use Example 2.1 Zoog as reference to determine the X,Y values. Try first. If needed, see the solution in the Exercise section (online)

//mouseX is a keyboard that the sketch replaces with the horizontal position of the mouse.

//mouseY is a keyboard that the sketch replaces with the vertical position of the mouse.

rect(mouseX, mouseY, 50, 50):

println(mouseX,mouseY);

Week #03

Chapter 3: Interaction-Review

Coordinate values stored in mouseX, mouseY or pmouseX, pmouseY can be applied to many properties. For example, to change the color, the brightness, or the dimension of an object

See examples [3.2 - 3.4 here](#)

Built-in Input Functions

Processing also has a number of input functions.

Among them are key and mouse input functions, also called event listeners, that continuously scan for the mouse and key for input activities and act accordingly. **mousePressed()**, **mouseClicked()** & **keyPressed()** are some of the popular ones ([see reference for more](#)):

** functions always have the (). Variables do not end with ()*

Interaction

Example 3.5 demonstrates implementations of both (input) functions and mouseX and mouseY variables.

- **void draw() is needed to repeatedly run the program**
- **void mousePressed() and void keyPressed()** are event listeners (functions). These functions **only execute the code when the mouse /key is pressed or clicked.**

Function mousePressed draws a square at the location mouseX and mouseY and keyPressed clears the screen

Exercise: Add a mouseClicked() function that will draw an ellipse at mouseX and mouseY

Lab #1

Copy and revise Exercise 3.4 such that it will draw a Zoog where the mouse is clicked and clear the screen with a keyPress

Chapter 4: Intro to Variables

Recall this toy car program:

Static values:

- The car is immobile
- To draw it at any different location and or different size, we need to manually determine and change many numbers, again and again

By using (system) variables mouseX and mouseY, instead of static numbers, we can:

- Drag and move an object(s) as the mouse moves.

And with input function, we can:

- Draw an object(s) at a new location when and where the mouse is clicked or pressed

Pre-defined/Built-in Variables

Processing also has a number of built-in variables (see reference here). Two of those are **width** and **height** that are used to store the width (x) and height (y) of the canvas.

** Unlike system variables, built-in variables can be redefined or updated. However, it is strongly NOT advised*

Take a look at example 4.5 where:

width/2 and **height/2** are used to determine the center coordinate of the canvas.

frameCount is a variable that counts the number of times the draw function is executed (as frame advances).

Using [built-in] Variables

// Consider this simpler version [copy and test run this program]

```
void setup() {  
    size(600,400);  
} // end of setup  
void draw() {  
    background(255);  
    // draw a circle at the center  
    ellipse(300, 200, 50, 50); // draw a circle at the center of the canvas  
} // end of draw
```

*if we change the size of the canvas, the circle will no longer be drawn at the center
Now replace with this (then change the size again)*

```
    ellipse(width/2, height/2, 50, 50);
```

Lab #2

Use built-in variables width and height, write a program that will draw four overlapping circles at the vertical and horizontal center, regardless of the canvas size, as shown. Use noFill or color opacity to show the overlaps

User-Defined Variables

// copy and test run this program]

```
int circleX = 300; //Declare circleX as type integer and assign 300
```

```
int circleY = 200; //Declare circleY as type integer and assign 200
```

```
void setup() {  
    size(600,400);  
} // end of setup  
void draw() {  
    background(255);  
    ellipse(circleX, circleY, 50, 50); // Draw a circle at circleX and circleY  
    ellipse(circleX+20, circleY, 50, 50); // Draw another circle at circleX+20  
} // end of draw  
// assign different numbers to circleX and or circleY for a different location,
```

So what is variable?

Generalization: variable is a symbolic name, a named pointer that points to a computer memory address, a location, where a piece of data is stored. It is a placeholder of a specific piece of data type

To implement user-defined variables, we have to:

1. Identify the data type
2. Declare the variable.
3. (optional) Assign initial value

```
int circleX; // declare type & var
circleX = 200; // assign value
// Or combine both into one statement
int circleX = 200;
```

Variable Types

Here are Processing's common **primitive data types**:

- **boolean**: for true or false value
- **char**: for any character
- **int** : an integer (whole) number
- **float**: decimal number, e.g. 3.1415
- **color**: RGB color value
- **long** : a large integer (two int)
- **double**: a large decimal (two float)
- **String**: a set of character(s). * **String** is a built in object, not a primitive type.

Before declaring a variable, we need to consider what data it will store and how it will be used. For examples: Social security numbers are not computational, so it is a string; Wage is a decimal number, so it must be a float (or double); X, Y locations can be either float or int

Naming Variables

Choosing an appropriate variable name ([see File & Variable Naming](#)):

- No reserved or predefined keywords such as if, mouseX, pmousedY, keyPressed, true, int, char, float, etc- [see reference](#)
- NO special characters, e.g., space, *, -, ?, / ' ; except underscore _
- Not begin with a number
- COMMON PRACTICE (not a requirement):
 - Use lowercase. If more than one word, capitalize the first letter of the next word. For example: **carColor**, **lastName**, **circleX**.
 - Use meaningful name
 - Avoid redefined system variables or built-in names, e.g, width, height.

Declaring Variables

```
char answer = 'y'; // Declare answer a char type and assign 'y' to answer
boolean happy = false; // Declare happy as boolean and assign false
int age = 10;      // Declare variable age of type int and assign 10 to age
float x = 4.0;    // Declare x as float and assign 4.0 to x
float y;         // Declare y as float
y = age + x;     //declared above, you can then change or assign value
```

More are available at ([4.1 : declaringvars](#))

Variables:

- Variables declared at the top and outside of program functions can be used anywhere in the program [more later]
 - Variables with fixed, non-volatile values are called CONSTANT. Constant is commonly defined using all CAP symbols. For example, HALF_PI, PI as previously described in Arc function.
 - Data Type is needed only when you declare the variable, not when you use it
 - An integer variable only stores integer value, a boolean variable is only for true or false...etc. Do not mix up types, e.g., int w = 3.1 or char ans = "yes"
 - Float can be used to store integer or float: float newnum = 10;
- Exercise:** Complete this [Variable Worksheet](#)

Using Variables

Exercise

Let's follow the steps below to implement variables in this toyCarFunction

- Declare variable carX, carY and initialize them to 200
- Replace x and y values of the car body with carX, carY:
rect(carX, carY, 120, 40);
- Change other X, Y using offset values
rect(carX+20, carY-30, 80, 30);
- Do the same for all other x, y, e.g.,
ellipse(carX+20, carY+40, 30, 30);
- Run the program again. You should see the exact same drawing
- To draw at different location, simply change the **initial values** of carX and or carY

Updating Variables

Use Example 4-2 for this exercise

1. Insert the following command in draw() (inside the { }) and run the sketch again
circleX= circleX + 1; **/* Do Not** enter int
2. As circleX increases, the circle moves toward the right.
3. Insert println(circleX, " "); on the next line and Run it
4. Replace circleX using circleY circleY= circleY + 1;
5. Try both
6. Try using different speed, including negative numbers

Lab #3

ToyCarMove. Use this toyCar program

- Add variable speed and assign number 1 to speed
- Update carX values by adding speed. This will move the car to the right.
Note that the wheels don't move. Add a command to move them. The entire car should move (out of the canvas).
- Add a keyPressed() input function that includes code to reassign ALL variables of X to their initial values as declared at the top. This will draw the car at the original location on keyPressed. **[SEE Next slide]**
- Run and press any key to make sure it works.
- Save it as toyCarMove– **You will need this for the LAB**

Lab #4

1. Revise this smileyFaceEmoticon program such that it **will draw a happy face anywhere with a mouseClicked**, i.e., **use mouseX and mouseY and mouseClicked function**. **Add a keyPressed() function to erase the screen** (see Example 3.5)*.
* It will not work unless you have a setup and draw function along with key and mouse events
2. Implementing as many variables as needed to replace **the static values** of this program.
 - Note that some variables can be reused multiple times.
 - Also use meaningful variable names

Week #04

Using Random Function

```
Int circleX = 100;
Int circleY = 100;
float bwColor = 0; // color of the circle
void setup() {
  size (480,270);
}
```

```
Void draw() {
  background(255);
  stroke(0);
  fill(bwColor);
  ellipse(circleX,circleY,50,50);
  // println(bwColor);

}
```

```
void mouseClicked() {
  bwColor = random(255);
}
```

Random is a function that returns a float number from 0 to 255, exclusive. When the mouse is clicked, bwColor is assigned a new random number that then changes the color (in draw function)

Study these: **Example 4.7 & 4.8-**

Exercise: Make the following changes to Example 4.2 (see prior slide) if needed:

- Add three variables on the top and assign three random color values.** Update fill function. Test run it.
- Add an input function, e.g., keyPressed(), mouseClicked() that will generate a new random number for each color.** Test run to see if color changes with every click or keypress

System Variables & Functions

We've learned and implemented many built-in functions. Each of those functions perform a specific task, e.g., draw or color something, but none returns any value. These type of functions are often called void functions or methods. Return function, as the name implies, returns a value to the function call.

Here are two that we've already implemented `random()`, `radians()`. Each of these returns a value that can be used accordingly. Other commonly used (math) functions include `abs()`, `int()`, `float()`, `constrain()`, `max()`, `min()`... More RETURN Functions are @: [See reference](#)

Return Function

Let's reuse (4.3).

if we want to restart circleX at a random location, we add the following mouse input function:
ADD the function below to 4.3 and RUN it!

```
void mouseClicked(){
    circleX = random(480);
} // end of mouseClicked
```

Since circleX is declared as an int data type, but random function returns a float, this will result in a type mismatch. We would need to change or convert the data type:

1. **convert (or type cast) float into an int as :** `circleX = int (random(480) );`
2. **Or declare circleX as a float:** `float circleX = 200;`

Assume we also want to restrict the ball's movement within the canvas, we must stop circleX from incrementing. The constrain function will do that

```
circleX = circleX+1;
circleX = constrain(circleX, 0, width); // insert this
```

If you want to stop it anywhere else, just change 0 (lower bound) or `width` (upper bound) to different values

[Save it as usingContrainFunction for HW](#)

Exercise

toyCarMove (Use your working toyCar or continue from last week)

- Add three color variables and assign the original RGB color values (red)
- We may already have a keyPressed() function. Add a mouseClicked function that will
 - Randomize the color of the car
 - Randomize the speed of the car– use variable speed

Note that random function return a float and our variables are integers, you may need to convert each to int
- Save it as nameToyCarMove.

Function & Variables

LAB #1: Write a program that mimics the circular downloading progress bar. The program repeatedly draws an incrementing arc to show the downloading progress until the circle (downloading) is completed. It will have random color. Here is the general idea (use the textual storyboard on the next slide for project planning):

- ◆ Draw an arc starting from 12:00 O'clock of arc length equals 1% (of 360)
- ◆ Repeatedly draws a new arc and increases arc length by 1 % each time
- ◆ Display number/percent showing current downloading percent inside the circle
- ◆ Stop drawing when the circle is completed (100% or 360 degrees)
- ◆ Add a mouseClicked() function to clear the screen and restart the drawing. Also generate a new random color

Lab: ([here](#))

Chapter 5: Conditionals

Relational Operators: To Compare TWO terms or expressions

- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to
- `==` equal to
- `!=` not equal to

`=` is the assign operator. DO NOT use it in the expression to compare

Logical Operators

- `&&` AND
- `||` OR
- `!` Not

X	Y	<code>&&</code>	<code> </code>	<code>!(X)</code>
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Simple If Statements

Exercise: Evaluate the following [expressions](#)

Syntax:

- `if (condition) // is true`
 ONE command to execute;

Condition is an expression that is evaluated to either true or false. They, however, can be used interchangeably

For more than one commands

```
if (condition) {  
    command 1 to execute ;  
    command 2 to execute ;  
    .....  
} // end if statement
```

Example:

```
int x = 100;  
int y = 50;  
//single command  
if (x > y)  
    ellipse(x, y, 50, 50); // end if  
if (x < y) { // 1 or more commands  
    rect(width/2, height/2, 35, 25);  
    triangle(25, 25, 50, 50, 25, 75);  
} // end if (x < y)  
println(x, " ", y);
```

Exercise: Revise [example 4.3 incrementingVar](#)) by adding a simple if statement to stop the ball at the boundary-- same as using `constrain()`

COMP 125 MIDTERM REVIEW

There will be multiple choices, fill in the blanks, short programming coding problems and questions from the labs and hw, so you DO need to know the implementation requirements and specifications in order to implement them correctly. I would recommend that you write important notes in a worksheet. **You will not have enough time if you keep searching for unorganized answers**

The **power points** give a broader and more comprehensive view than this shortened guideline. So please go over them and learn the **coding examples**, including the **keywords and concepts**. Topics to review:

Chapter 1-2: Pixels and Processing

- **X and Y coordinates**
- **Using 2-D primitive functions** (basic drawing functions)
- Using basic format and environment functions, **fill()**, **stroke()**, **background()**, etc.
- Identify and implement **RGB color, RGB color number system**

Chapter 3: Interaction

- Using built-in and system variables: **mouseX**, **mouseY**, **pmouseX**, **pmouseY**, **width**, **height**
- Using and coding input functions: **keyPressed()**, **mousePressed()**, **mouseClicked()**

Chapter 4: variables and datatypes

- Identify and using basic data types: **int**, **float**, **boolean**, **String**
- **Variable declarations**
- Using variables: **incrementing**, **decrementing**, **updating to animate objects**
- Using system variables: **width and height**
- Using **random()**, **int()**, **float()**, **abs()**, **constrain()** functions

Chapter 5: Expressions and Conditionals

- **Use boolean variable**
- Identify relational (<, <=, >, >=, ==, !=) and logical (&&, ||, !) operators
- Evaluate an expression/condition to **true or false** (ref: if worksheet)
- Evaluate and construct **if**, **if else**, and **compound if statements**

Chapter 6: Loops (TBA) *HAVEN'T LEARNT IT YET!!!

- Understand for loop & while loop (syntax, implementation purposes, and requirements)
- Determine the output by tracing each of the steps in the for and while loops [Worksheet]
- Write the code to construct and convert between for and while loop
- Identify infinite loop

Week #05

If Else (Only if time permitted)

Analyze the following: ([here](#))

Exercise

- Revise example 4.3 by implementing an **if else statement** and necessary code if needed so that the ball restarts on the left after it hits the right

Exercise 1 & 2:

- compare2Num
- coinFlip

5.2 - Conditional Quadrants ([here](#))

5.3 - Rollovers ([here](#))

5.4 - Hold Button ([here](#))

5.6 - Bouncing Ball ([here](#))

5.8 - Path Along Edges ([here](#))

5.9 - Gravity ([here](#))

5.10 - Bouncing Zoog ([here](#))

stopNGo ([here](#))

Lab #1 - dayOfTheWeek

Lab #2 - Rock, Paper, Scissors ([here](#))

Boolean

Study and run the following examples:

1. 5.4 and 5.5: Hold and Switch Button

Exercise:

IF stopNGo

Lab #3 - LightOnOff (#1)

Lab #4 - LightOnOff (#2)

Week #06

Chapter 6: iterations-- Loops

In processing, draw() function is an infinite loop. Run this

```
int x = 0;
void draw() {
  x = x + 1;
  println(x);
}
```

To repeat a set of commands a limited number of times, we use loops. The two common loops are:

1. while loop
2. for loop

example 6.1 to 6.3

Chapter 6: iterations-- Loops

Syntax: While loop is constructed as an if statement. The loop only executes when the expression is true. Thus the expression must change to false to stop the loop:

```
while (condition) // true/false
{
  commands(s)
  update (condition*)
}
```

Let's run this & change x

```
int x = 0;
while (x < 100) {
  rect (x, 50, 10, 10);
  x = x + 20;
}
```

While Loops

Evaluate the following:

```
size(200,200);  
int x = 50;  
// if only executes once if true  
if (x < width) {  
    x = x + 50; //  
    rect(x,100, 20, 10);  
} // end if  
println(x);
```

Vs this:

```
size(200,200);  
int x = 10;  
// loop stops only when x ==> y  
while (x < width) {  
    x = x + 50; //  
    rect(x, 100, 20, 10); //  
} // end while  
println(x);
```

While Loops: Counter Control

// counter control

```
int x = 20;  
int count = 4;  
while (count > 0) {  
    rect (x * count ,50, 20 ,20);  
    count --; // count = count -1;  
} // end while  
println(count);
```

- Try then change count < 0
- Then change count++

// event control

```
int x = int(random(50 ,100));
```

```
while (x > 60 && x < 90){  
    println(x, "degrees. It is a nice day");  
    x = int(random(50,100));  
}  
println(x, " degrees. It is not a good day");
```

For Loop

For loop is used when we computationally know how many times the loop repeats. Study the following programs

- Example 6.6, 6.10, & 6.11

* for loop is also known as counter control loop whereas while loop is an event control loop.

Here is the Syntax:

```
for (initial value; exit condition; update) {  
    command(s)  
} // end of for loop
```

The three expressions are in order
and separated by a semicolon (;)

1. Starting value of counter variable
2. Exit condition (when expression is false)
3. update the counter value

//Example 1: Test run.

```
size (400, 400);  
int incr = 40;  
int w= 10;
```

```
for (float y = 0; y < height ; y =y+incr) {  
    rect(0, y, y+w, 20);  
} // end of for y
```

//Example 2 & 3

```
size(400,400);  
int x = 100;  
int y = 200;  
// repeats 5 times, from 0 to 9, increases by 2  
for (int start = 0; start <10; start +=2) {  
    rect(width/2,10,x,y);  
    y -=20; // decrement y  
} // end of for start
```

```
// and test run this –  
//for (int x=1; x<width; x++){  
//  line(10,10,random(x), random(x));  
//}
```

For Loop

```
// The bad for loop: Fix this  
size(100,100);
```

```
for (int x = 0; x == width; x += 30)  
{  
  fill(255);  
  rect( x,100 ,20,50);  
}  
fill(0);  
rect(50,30,100,100);
```

```
// Another bad loop  
size(400,400);
```

```
for (int x = 0; x >= 0; x +=20)  
{  
  fill(s);  
  rect(x,100,5,50);  
}  
fill(0);  
rect(50,30,100,100);
```

Assignments

Exercise:

Tracing for loop

Complete these loop exercises (2)

Lab #2- Modify these tenWhealers by implementing a for loop to draw the wheels as described.

For Loops

Lab #3- Concentric circles: Draw a concentric circle similar to ones shown aside. Start with the outer circle and reduce the width of each circle as it loops to draw the smaller inner ones. This will place the smaller circles on the top of the big ones. Use any color combinations and changes you want inside the loop.

* Optional: Use another loop to draw the second concentric circle. Make sure to change circleX to draw it at a new location

Submission

1. Lab 1: tenWheelers (while loop)
2. Lab 2: tenWheelers (for Loop)
3. Lab 3: Concentric rings

Week #07

Global vs Local Variables

Trace to analyze this example. Try each of these individually to understand how x changes

1. Replace draw line to:
ellipse(x,height/2,20,20);
2. Modify the mousePressed() to display x
3. Move int x = 0; to top
4. Add x = 0; where int x = 0; was
5. Global variables are declared at the top. They are available/accessible everywhere
6. Local variables are defined within/inside a code block such as function, loop, if, etc.
Local variables are only available inside that specific code block
7. **See Example:** Global and local variables

Nested Loops

A nested loop is a loop that appears in the body of another loop. The inner and outer loop can be any while or for loop combinations. For each pass of the outer loop, the inner loop must repeat the entire sequence. Thus a nested loop repeats an $\text{inLoop} * \text{outLoop}$ times

```
int w = 0;
while (w < 3) {
    print(w, ": "); // command in outer loop
    // inner loop
    for (int i = 0; i < 5; i++){
        print(i, ", "); // command in inner loop
    } // end of inner for loop
    w++; // command in outer loop
    println(); // line break
} // end of outer loop
```

1. Implementation Example
2. **Exercise 1:** Consider drawing the 3 by 3 tic-tac-toe board as shown aside. We can use these approaches:
 - a. Draw 9 squares or two lines on each direction. We need to repetitively and evenly space/arranged them
 - b. Repeatedly drawing a square three times. Then repeat the first step 3 times

Here is the template to draw a 3 by 3 tic-tac-toe (if you need one)

Nested Loops

LAB #1: Consider a scenario when we need to draw many objects inside each of the randomly located objects, e.g., random dots inside random squares. With prior knowledge, this problem can be accomplished using a nested loop that includes an outer while loop. Write that program. Here is the template if needed

Chapter 7: Function

Generalization: Function is a named code block that can be invoked at any point in a program to perform a specific task. There are several, but two of the main purposes and advantages of using function are:

- **Reusability:** Function allows the reuse of code to repeatedly perform a specific operation
- **Modularity:** Function provides a mechanism to break down a larger and complex task into smaller and simpler tasks, thus making it easier for organizing and managing tasks

Built-in Functions

We have already used many of these built-in functions (no declaration needed). They can be used and reused as applicable in the program. For example:

```
rect(); fill(); abs();
```

And these two that we have to explicitly redefine

```
void setup() {... }  
void draw() {...}
```

We use setup and draw functions to separate code into two specific tasks. One for setting and the other (draw) to repeatedly execute code

User-Defined Function

Often times, we have to write own functions. Here is an example: [Example 7-2: Calling a function](#)

```
void setup() {  
    size(200,200);  
} // end setup  
  
void draw() {  
    background(255);  
    drawBall(); // call/invoke the function  
} // end draw  
void drawBall() { //declare user defined function  
    fill(0);  
    ellipse(10,100,20,20);  
} // end drawBall
```

Note: drawBall function is repeatedly invoked inside draw(); however, since x and y are always 10 and 100, the balls remain at the same location.

Exercise:

1. Declare int variable x = 10 and speed =1. These are **GLOBAL Variables**
2. Replace 10 from ellipse with variable x. Run to verify it works
3. Write a move function that increases x by the value of speed. Call to execute move function to move the ball.
4. Write a bounce function to reverse the ball

Planning & Modularity

Example: Bouncing ball

In addition to setup and draw(), we want to separate the program code into smaller functions of specific tasks

- Drawing
- animation
- drawBall(): to draw a ball
- **Animation**
 - Move ball(): move to next location
 - Bounce(): Reverse the ball directions when it touches the edges

Once we have the structure, we then can list the variables, especially the common ones that are used in various functions

Assignments: User-Defined Function

Exercise: Bouncing Zoog

Study and reorganize Example 6.10 into functions. Consider these:

1. Draw Zoog
 - a. How many functions would you use?
2. Animate Zoog
 - a. What are the activities that can be separated?
3. main/draw: write function calls and other the remaining code

Assignments: User-Defined Function

Lab #2: toyCarFunctions

Use the (last) toyCarMove program. Save a copy as toyCarFunctions and modify it to include

1. A drawToyCar function: You may consider breaking this into smaller functions
2. A carMove function
3. A carDirection (bouncing) : change car direction if it touches the edges
4. main/draw: write function calls and other the remaining code

Function Parameters & Arguments

```
void setup() {  
    size(200,200);  
    drawBall(); // function call/invoke  
    drawBall(); // function call/invoke  
} // end setup
```

```
void drawBall() { // declare function  
    fill(0);  
    ellipse(10,100,20,20);  
} // end drawBall
```

Consider the drawBall function [left]. Since static values are used, the function draws the same circle - exact ones on top of others, regardless how many times you call it.

```
float todayTemp = 100.00;
```

```
void setup(){  
    celConverter( 45);  
    celConverter( 32 );  
    celConverter( todayTemp );  
}
```

```
void celConverter( float f ) {  
    float c = (f - 32) * (5.0 / 9.0);  
    println(f, "F degrees = ", c, "C degrees");  
}
```

By declaring variable **float f** in the function declaration statement, different values can be converted, thus allowing celConverter function to be reused as needed

Consider this example:

```
int rectW= 20;
```

```
int rectH = 10;
```

```
void setup(){  
    rectArea(rectW, rectH);  
    rectArea(rectW*2, rectH);  
    rectArea(100, 50);  
}
```

```
void rectArea( int w, int l ) {  
    println("Area of rectangle", w, "and",l," is ", w*l);  
}
```

Note that since we declare **int w, int l** in the function header, rectArea function is only capable of receiving two integer values

Function Parameters & Arguments

Exercise: the drawBall(float ballX) only accepts x. Revise the function that will accept any y value thus allowing drawing different balls at any x and y location

LAB #3: sumOf3Nums

- A. Write a program (revise rectArea program) that includes a function that receives any three numbers as parameters. It then adds and displays the sum of the three numbers.
- B. Write several function calls in the setup function using different values and variables to test the function.

Function Parameters & Arguments

LAB #4/HW: lightBulbs

1. Revise this lightBulb program that includes a drawLighBulb function using two parameters, bulbX and bulbY. Write several function calls draw (or setup) with different values to draw several light bulbs at different locations

Week #08

1. Midterm #2 (10/ 31)

- Go over the Quizzes and exam 1 on Sakai
- Go over PPT and Labs. Here is the [Topical Review](#)
- **Requires Respondus LockDown Browser.**

2. [Programming Project #2](#) : Due on 11/05

3. HW/Labs (Questions?)

Function Parameters (continue)

Exercise: The [sunFlower program](#) draw a static sunFlower regardless how many times it is called

To draw the flowers of different sizes, we need to implement a function that has a **parameter** that accepts the flower size

- Copy and paste the entire drawSunFlower function to the bottom of the program. We now have a second drawSunFlower function
- Add a parameter, float fSize, to the second as shown
 - void drawSunFlower(float fS) {
- Change all 50 to fS in the new function
- Add a new function call in draw() and send in any value, e.g, drawSunFlower(40);
- Run and change the value to verify

Exercise: Arguments & Parameters

Similarly, to draw the flower at different locations, we need parameters for the coordinates. Let's add another drawSunFlower function

- Copy and paste the entire drawSunFlower() to create a new drawSunFlower function
- Add a parameters float x, float y to the function
- void drawSunFlower(float fx, float fy)
- Change the location values to fx, fy in the new function
- We can now
 - Write a function call in draw() and send in any fx and fy value, e.g, drawSunFlower(100, 200);
 - Declare a global variable call flowerX and flowerY and initialize them to any values you want
 - Write another function call and pass in the variables: drawSunFlower(flowerX, flowerY);.
- Run and change the variable flowerSize's value to verify

Function Overloading

We now have three drawSunFlower functions:

1. The first and original function draws a flower of size 50 at the center of the canvas
2. The second function draws the flower also at the center, but it allows you to specify the flower size
3. The third function allows you to specify the location of the flower.

The concept of using functions with the same function name but having different parameters is called **function overloading**. **Overloaded functions must have different number of parameters and or completely different data type. Float and int are not the same!**

LAB #1

1. None of the three drawSunFlower functions can draw a flower at different location and different size. Overload another drawSunFlower function that takes four parameters to draw a flower at any location (x and y) of any size(w and h). Write several function calls to test this new function (comment out previous function calls to make sure it works)
2. Add another function, growFlower(float changeW, float changeH). This function takes two parameters (float changeW and float changeH) and to increase or decrease the flowerW and flowerH, e.g.,
 flowerW += changeW;
 flowerH += changeH;
 - a. *flowerW and flowerH should be declared as **global variables** (if not already)*
 - b. *Add an if statement to stop resizing if flower is already too big or too small*
 - c. *Add a growFlower function call in draw with two arguments to change w/height*
3. Invoke growFlower in draw function to animate it

Return Function

Consider the following built-in functions:

```
abs(x); // returns the absolute value of X
random(x); // returns a random number from 0 to x
int(x); // converts x into integer and returns it
```

When invoked, these functions **return a value**, thus called **return function**.

Void Function VS Return Function

Declaration of a void function

```
void functionName (dataType parameter*)
{
    // function code
    // # has NO return
}
```

- Void functions are defined as **void** and has no return statement.
- Parameters are optional

Declaration of a return function

```
dataType functionName (dataType parameter*) {
    // function code
    return variable; //same return dataType
}
```

- instead of void, **return function has a declared dataType that matches with the return dataType.**
- Parameters are optional, but often needed

Void Function VS Return Function

Example of void function

```
float c ;
void setup(){
    flipACoin(); // void function call
}
void flipACoin() {
    c = int( random(2)); //return function call

    if (c == 1)
        println(c, "It is head");
    else
        println(c, "It is tail");

}
```

Example of return function

```
float c ;
void setup(){
    if (flipACoin()== 1) // return function call
        println("It is head");
    else
        println("It is tail");
}

int flipACoin() { // return function declaration
    int c = int(random(2)); // return function call
    return c;
}
```

User Defined Return Functions

Example of return function

```
int a=10; // global variables
```

```
int b = 5;
```

```
void setup () {
```

```
    float quo = quotient (50, 2);
```

```
    println("The quotient is : ", quo);
```

```
    println("The quotient of : ", a , " and ", b,
```

```
           " is ", quotient(a,b));
```

```
}
```

// Example of return function

```
float quotient (float dividend, float divisor) {
```

```
    float result; // result Must be a float
```

```
    if (divisor == 0)
```

```
        result = 0;
```

```
    else
```

```
        result = dividend / divisor;
```

```
    return result; // return a float
```

```
}
```

Return function: Example 7.4

```
void setup () {  
  size(480, 270); }  
void draw() {  
  background(255);  
  stroke(0);  
  float d = distance(width/2, height/2, mouseX, mouseY);  
  fill(d*3, d*2, d);  
  ellipseMode(CENTER);  
  ellipse(width/2, height/2, 100, 100);  
} // end of draw()  
float distance(float x1, float y1, float x2, float y2) {  
  float dx = x1 - x2;  
  float dy = y1 - y2;  
  float d = sqrt(dx*dx + dy*dy); // local d  
  return d; } // end of function distance  
}
```

User Defined Return Function

Exercise: comparing two numbers

```
int a = int(random(10));
int b = int(random(10));
int c = int(random(10));
int d = 7;
void setup(){
    print(a,b,c) ; // test code
    int largest = maxNum(a, b); // function call
    println("The larger number is : ",
        largest ); // will output
    if (maxNum(a,b) >= d ) // function call
        println("greatest is: ",_____ );
    else println("greatest is: ", d );
}

}

// your maxNum function goes here
```

Fill in the blanks to complete the maxNum function. This function takes any two int parameters, a, b and returns the greater number of the two.

```
_____ maxNum ( int x, int y) {

    _____ largest; // return variable-- local

    if ( x >= y)
        larger = x;
    else
        larger = y;

    _____; // return statement
```

User Defined Return Functions

// examples of RETURN functions

```
int a = int(random(10));
```

```
int b = int(random(10));
```

```
int c= int(random(10));
```

```
int total;
```

```
void setup () {
```

```
    total = sumNum( a,b); // example
```

```
    println("The sum of ",a, b," is ", total);
```

```
    // second example
```

```
    if (sumNum(a, b) > sumNum(a, c) )
```

```
        print( b, "is greater than ", c);
```

```
}
```

Exercise: Add the sumNum function that takes two integer parameters and returns their sum to the function call.

Boolean Function

Exercise: Function can also return true/false. Complete this program and boolean isEven function that, given an integer value, it returns true if the number is an even number and false otherwise

```
void setup(){ // add the code below to void setup
    // sample test code
    int num = int(random(100) );
    if (isEven(num))  // ( isEven(num) == true)
        println(num, " is an even number ");
    else
        println(num, " is an odd number ");
} // end of setup
// Complete the isEven boolean function below
```

```
boolean isEven( int num) {
    boolean even = false;
    int qRemain = num % 2;
    if (qRemain == 0)
        even = true;
    //else
    //    even = false;
    return even;
}
```

Lab #2 In class demo: Using this gravity program

Modify to use a boolean function that detects basic collision of moving objects

Return Function

Lab #3 In class demo: Using this gravity program

Modify to use a boolean function that detects basic collision of moving objects

User Defined Return Function

Lab #4/HW: Do #3, maxOf3 function

Assignments/Submission

1. Lab #1- Lab Demo: function overloading
2. Lab # 2- Lab Demo: Boolean function
3. Lab # 3/HW: Return Function, maxOf3 Prob #3
4. Lab # 4/HW: boolean workDay function: Do #4

Also

Start working on the project!

Prepare for Midterm #2

Week #10

Chapter 8: Object Oriented Programming (OOP)

OOP is a programming style that employs a structure called classes and objects.

To program in OOP, we need to create a class, if one not exists. **Class**, generally, is a definition or description of thing, whereas **object** is a piece of data created from class. **A class declaration has four key sections:**

1. **Class name** - class declaration. Class name begins with a capital letter
2. **Data members**/variables: Properties associated with each object
3. **Constructor(s)** - functions used to create instance(s) of the object
4. **Member functions**, also called Object/member methods. Including "special member functions, i.e., constructors, accessors and mutators) : define how and what can be performed on each object

OOP: The Car class- Example 8.1

```
// #1. class definition. Class name is case sensitive
class Car { //different from fileName
//#2. data fields, properties of the (all) car object
color c;
float carX;
float carY;
float speedX;
// #3. default constructor- same as class name
// It is a function without void or return
Car() { // to create a car object and initialize variables    c= color(255,0,0);
    carX = width/2;
    carY = height/2;
    speedX = 1;
} // end default constructor

// #4 member functions-
// define object's functionalities
void display(){
    rectMode(CENTER);
```

```

        fill(c);
        rect(carX, carY,20,10);
    } // end display

void moveForward(){
    carX = carX+speedX;
} // end moveCar

} // END CLASS

```

Week #11

OOP: Multiple Classes

- Programs often use more than one objects from more than one classes. Let's go over these examples
- **Exercise:** We will recode this School bus program using objects:
 - Write the Bus class.
 - Create bus object and test run the program
 - Add people class
 - Add driver using People class and test run
 - Add passengers

Note that in this program:

- There will be two Objects/Classes
 - People - easily add more people
 - Bus
- Layout/arrangement
 - Bus can be drawn anywhere
 - People are inside the bus
- functionalities and implementation (ON YOUR OWN)
 - People and bus move synchronously

- Bus restarts after passing width (same for People)

Lab #1/HW: Complete this Light Bulb program using the OOP Light Bulb program template as described here. You are also free to write your own program. Generally:

- Two Classes
 - Bulb
 - Button/Switch
- Layout/arrangement
 - We can have multiple sets of Bulbs and Buttons
 - Each Button controls a specific Bulb
- functionalities
 - On mouseClicked:
 - If the user clicked on a button, change bulb color (on to off and vice versa) and the bg color to dark and bright and vice versa

Chapter 9: Array

In an office building, the entire floor is typically divided into offices or rooms and each room is labeled as 1,2,3... So 105, 311 are typically room number 5 on first floor and room number 11 on the third floor. Similarly:

- To store data, memory space is partitioned and indexed sequentially starting from **0** to n-1, where n is the total number of partitioned memory spaces
- This type of structure, or data representation, is called **Array** and the number sequence used to label each space is the **index** of the array

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Array

To declare array:

```
int[ ] y = new int[10]; // declare an array for 10 integers
```

```
float [ ] x = new float[5]; // declare an array for 5 float numbers:
```

Note that:

Array declarations only partition and reserve the memory for storing the data. No values are assigned.

Array is data specific, i.e., an int array can ONLY be used to store integer numbers, a String array can be used to store any text strings

Array: Initialize elements of an array

- `int [ ] y = new int[3];` // declare an integer array of 3 elements

--	--	--

`y[0] = 5;` // (store) initialize each specified element individually. Data type must be the same
`y[2] = 100;`

5		100
---	--	-----

- You can declare and initialize all elements of the array together in one command. Processing will assign the index number for each element of the array
`int [ ] y = { 5, 100, 20 };` // note that `new int[3]` is not required

5	100	20
---	-----	----

`float [ ] x = { 1.2, 3, 2, 3.4123, 9.9 };`

1.2	3.0	2.0	3.4123	9.9
-----	-----	-----	--------	-----

Examples (Chapter 9)

1. Declare arrays: Exercise 9.1-3:
2. **Exercise 1:** Using array:
 - a.
 - i. Declare an array and assign any three (3) integers
 - ii. Add each element of the array to the total
 - iii. Print the total using println
 - b. Copy (a) and revise:
 - i. Reassign each element to a random value from 0-100
 - ii. Display each random number from the array and their sum

Extra Exercise (if time allowed)

Exercise: Write a program that uses two integer arrays , circleX and circleY, of three elements and assign three valid numbers to each of the two arrays. Draw three circles at the locations as assigned in circleX and circleY arrays. Add a mouseClicked function to generate new random numbers for each element in the two arrays. This will draw three circles at different locations.

Week #13

Chapter 15: Working with Images

Processing supports several different image types, including the popular gif, jpg, and png

In Processing, image object or image data type is defined as **PImage**

Below is the syntax to declare an image object variable :

```
PImage imageVarName;
```

Note that **PI** must be capitalized

Once a PImage variable is declared, you can use loadImage command to load the image into the variable as:

```
imageVar = loadImage("fileName.jpg");
```

Note that in the example on the right, the images must be inside the “data” folder where the program file, sideBySide, is as shown . **Do Not create another folder inside the data folder**

Working With Names

For this unit, you need the images from [this Images folder](#).. Download and **Extract** them.

*Copy and Run the program [right]. It will create a folder , "data" Inside the program folder.
Copy the animal image files into the “data” folder.*

Run it again

```
PImage img;
String imgName= "animal0.jpg"; void setup() {
    size(520, 440);
    img = loadImage(imgName);
}

void draw() {
    background(0);
    imageMode(CENTER);
    image(img, width/2, height/2);
}
```

randomAnimal: Array of fileNames

Exercise #1: Make sure to have a “data” folder with animal images inside the program folder

```
PImage img;
// array of animal image filenames
String[] animalArray = {"animal0.jpg", "animal1.jpg", "animal2.jpg", "animal3.jpg",
"animal4.jpg", "animal5.jpg", "animal6.jpg", "animal7.jpg", "animal8.jpg", "animal9.jpg"};

// generate a random number from 0 to the length of the array

int idx = int(random(animalArray.length));
```

randomAnimal: Array fileNames

```
void setup() {  
    size(520, 440);  
    img = loadImage(animalArray[idx]); // load the initial image  
}  
void draw() {  
    background(0);  
    imageMode(CENTER);  
    image(img, width/2, height/2);  
}  
void mouseClicked(){  
    idx = int(random(animalArray.length)); // generate new index  
    img = loadImage(animalArray[idx]); // load the new image based in the index  
}
```

randomAnimal: String Concatenation

Exercise #2: A different approach is to concatenate strings to form filenames

PImage img;

```
String fileName= "animal";  
// generate a random (image) number  
int idx = int(random(10)); // for 10 pics
```

```
void setup() {  
    size(520, 440);  
    // concatenate strings into image fileName  
    img = loadImage(fileName+str(idx)+".jpg");  
}
```

```
void draw() {  
    background(0);  
    imageMode(CENTER);  
    image(img, width/2, height/2);  
}
```

```
void mouseClicked(){  
    // to load a new random animal image  
    idx= int(random(10));
```

```
img = loadImage(fileName+idx+".jpg");  
// Processing implicitly converts int to string. Recommend using str(idx)  
  
}
```

randomPlayCard Image

LAB #2: Copy randomPlaycardName (LAB #1) and rename it as randomPictureCard. Revise the program by including the extension “.jpg” to form a fileName corresponding to the image card filename and the necessary code to display the image of that playcard.

1. Run it. This will create a folder, namely, randomPictureCard
2. Copy **ONLY the playcard images** and put them inside the “**data**” **folder** in the randomPictureCard.

Pixelate PImage

Exercise (similar to 15.7)

Make sure to have data folder with image

```
PImage img;  
void setup() {  
    size(200, 200); // MATCH Image size  
    img = loadImage("sunflower.jpg");  
    drawPixels();  
}  
void drawPixels() {  
    loadPixels(); //Create RGB pixels array  
    img.loadPixels(); //load image into pixels array  
    pixelateImage();  
    updatePixels(); // update pixels array (canvas)  
}
```

```

void pixelateImage() {
    for (int i = 0; i < img.pixels.length; i++ ) {
        // pull out the color properties from a pixel
        float r = red(img.pixels [i]);
        float g = green(img.pixels[i]);
        float b = blue(img.pixels[i]);
        float a = alpha(img.pixels[i]);
        // store each image's pixel to default array
        pixels[i] = color(r, g, b);
    } // for loop
} // pixelateImage

```

Array of Pixels

- Each pixel element uses 3 bytes: r, g, b and sometimes the 4th byte for alpha.
- The `img.loadImage()` function stores each image pixel into a single dimensional array. the pixels array.

i=0 1 2 width*height

r,g,b,(a)	r,g,b,(a)	r,g,b,(a)		
-----------	-----------	-----------	------	--

- **img.pixels.length** returns the length (size) of the array
- A for loop is used to traverse and pull out each of the color properties of each pixel from the array. The values are then assigned to the default pixels array [canvas] through `pixels[loc] = color(r, g, b);`
- `updatePixels()` is then used to update the all pixels displayed in the canvas. This is not required unless the pixels properties are modified

Manipulating pixels

- Add the println below into the pixelate function [previous slide] to output the values

```
println(r, " ", g, " ", b, " ", a, " ", brightness(pixels[i]));
```

- We can manipulate individual colors in a pixel in many ways. Below are some examples :

Replace and Test run each (separately!

```
//pixels[i] = color(g,b,r); //– original color  
pixels[i] = color(r,g,b, 0);  
pixels[i] = color(r*2, g/2, b/2);  
pixels[i] = color(random(r), random(g), random(b));
```

